



## University of Sialkot

Department of Software Engineering

### Word Analyzer

*Semester Project Report*

**BSSE -VI    Blue    Fall 2023**

**Submitted by:** Abdullah Tanveer (23010101-061) & Muhammad Bilal (23010101-079)

**Submission Date:** 7-July-2026

**TOTAL GRADE POINT**

**10**

**GRADE POINT  
ALLOCATED**

**ASSESSOR'S COMMENTS**

**Instructor: Museb Khalid**

# Deliverable 1: Client-Side Architecture

## INTRODUCTION

Word Analyzer is a full-stack web application that lets a user paste or upload a piece of text and instantly receive a detailed statistical, structural, and AI-generated breakdown of it. The goal of the project was to combine deterministic text analysis (character counts, word frequency, reading time, etc.) with generative AI features (summarization, grammar checking, tone detection) in a single, clean interface.

This deliverable focuses exclusively on the client-side of the application. The goal was to fully architect and implement the frontend using Vue 3 and Vite, with a modular component structure, centralized state management via Pinia, and a clean service layer for all communication with the backend API.

## TECHNOLOGY STACK

The client application was built with the following technologies:

- **Vue 3 (Composition API with `<script setup>`, TypeScript)** → core UI framework
- **Vite** → development server and build tool
- **Vue Router 4** → client-side SPA routing with lazy-loaded views
- **Pinia** → centralized state management (analyzer store, ai store)
- **Tailwind CSS** → utility-first styling
- **Chart.js (via `vue-chartjs`)** → word frequency bar chart and character distribution doughnut chart
- **jsPDF + jspdf-autotable** → client-side PDF report export
- **Axios** → HTTP client for all requests to the Express API

## 1. SINGLE PAGE APPLICATION ROUTING

Vue Router is used to define the two top-level routes of the application. The Home route loads eagerly, while the Analyzer route is lazy-loaded via a dynamic import so it is only downloaded once the user actually navigates to it, keeping the initial page load smaller. Navigation between views uses `router.push()` from within component methods rather than `<RouterLink>`, and a global `scrollBehavior` resets the scroll position to the top on every navigation.

| Route    | Purpose                                                                                                     |
|----------|-------------------------------------------------------------------------------------------------------------|
| /        | Home page — introduces the analyzer and links into the tool                                                 |
| /analyze | Analyzer workspace — text input, file upload, statistics, charts, AI panel, and feedback form (lazy-loaded) |

```

src > router > TS index.ts > router > routes
1  import { createRouter, createWebHistory } from 'vue-router'
2  import HomePage from '../views/HomePage.vue'
3
4  const router = createRouter({
5    history: createWebHistory(),
6    routes: [
7      {
8        path: '/',
9        name: 'home',
10       component: HomePage
11      },
12      {
13        path: '/analyze',
14        name: 'analyzer',
15        component: () => import('@views/AnalyzerPage.vue')
16      }
17    ],
18    scrollBehavior() {
19      return { top: 0 }
20    }
21  })
22
23  export default router

```

*Router configuration (client/router/index.ts)*

## 2. MODULAR COMPONENT ARCHITECTURE

Advanced Word Analyzer follows a modular component architecture in Vue 3. The application is divided into reusable components, top-level views, and a service/store layer instead of writing all logic in a single file. This structure improves readability, testability, and separation of concerns between presentation and business logic.

### components/ (Reusable Components)

- **AIPanel.vue** → renders the six AI feature cards and the free-form Ask-AI box; dispatches actions to the ai Pinia store
- **WordFrequencyChart.vue** → renders the horizontal bar chart of the top 10 most frequent words using Chart.js
- **CharacterDistributionChart.vue** → renders the doughnut chart of character-type distribution (uppercase, lowercase, digits, spaces, punctuation, special)
- **FeedbackForm.vue** → star-rating and comment form; posts directly to the feedback API on submit

### views/ (Main Pages)

- **HomePage.vue** → landing page with a live annotated text sample and a call-to-action into the analyzer
- **AnalyzerPage.vue** → the main workspace — orchestrates text input, file upload, all result sections, and the four components above

### Core Files

- **router/index.ts** → defines application routes and navigation logic using Vue Router
- **App.vue** → root component; contains the layout shell and <RouterView />
- **main.ts** → entry point of the Vue application; initializes and mounts the app with Pinia and the router installed
- **services/api.ts** → centralized Axios instance used by every store for backend communication
- **services/textAnalyzer.ts** → client-side text utilities — word search, highlighting, and duration formatting
- **services/pdfExport.ts** → builds and downloads the analysis report as a PDF using jsPDF
- **stores/analyzer.ts** → Pinia store holding the input text and all analysis results returned by the API
- **stores/ai.ts** → Pinia store holding AI feature results, loading state, and the Ask-AI question/answer

### 3. STATE MANAGEMENT & INTER-COMPONENT COMMUNICATION

Rather than passing analysis data through many layers of props, the application centralizes state in two Pinia stores. AnalyzerPage.vue reads store state reactively via storeToRefs() and passes only the specific slice of data each child component needs — for example, WordFrequencyChart.vue receives just the words prop, and CharacterDistributionChart.vue receives just the data prop, rather than the entire store. AIPanel.vue and FeedbackForm.vue similarly receive a text or wordCount prop from the parent view, and internally call their own store actions (or, in FeedbackForm's case, the API service directly) rather than emitting events back up — keeping data flow simple and mostly one-directional.

```
20 export const useAnalyzerStore = defineStore('analyzer', {
52   actions: {
53     async analyze() {
54       if (!this.text.trim()) {
55         this.fileError = 'Please enter or upload some text first.'
56         return
57       }
58       this.fileError = ''
59       this.isAnalyzing = true
60
61       try {
62         const { data } = await api.post('/analyze', { text: this.text })
63         if (data.success) {
64           this.basicStats = data.data.basicStats
65           this.characterAnalysis = data.data.characterAnalysis
66           this.wordAnalysis = data.data.wordAnalysis
67           this.keywords = data.data.keywords
68           this.readingAnalysis = data.data.readingAnalysis
69           this.hasAnalyzed = true
70           this.runSearch()
71         }
72       } catch (err: any) {
73         this.fileError = err?.response?.data?.message || 'Analysis failed. Is the backend running?'
74       } finally {
75         this.isAnalyzing = false
76       }
77     },
78   },

```

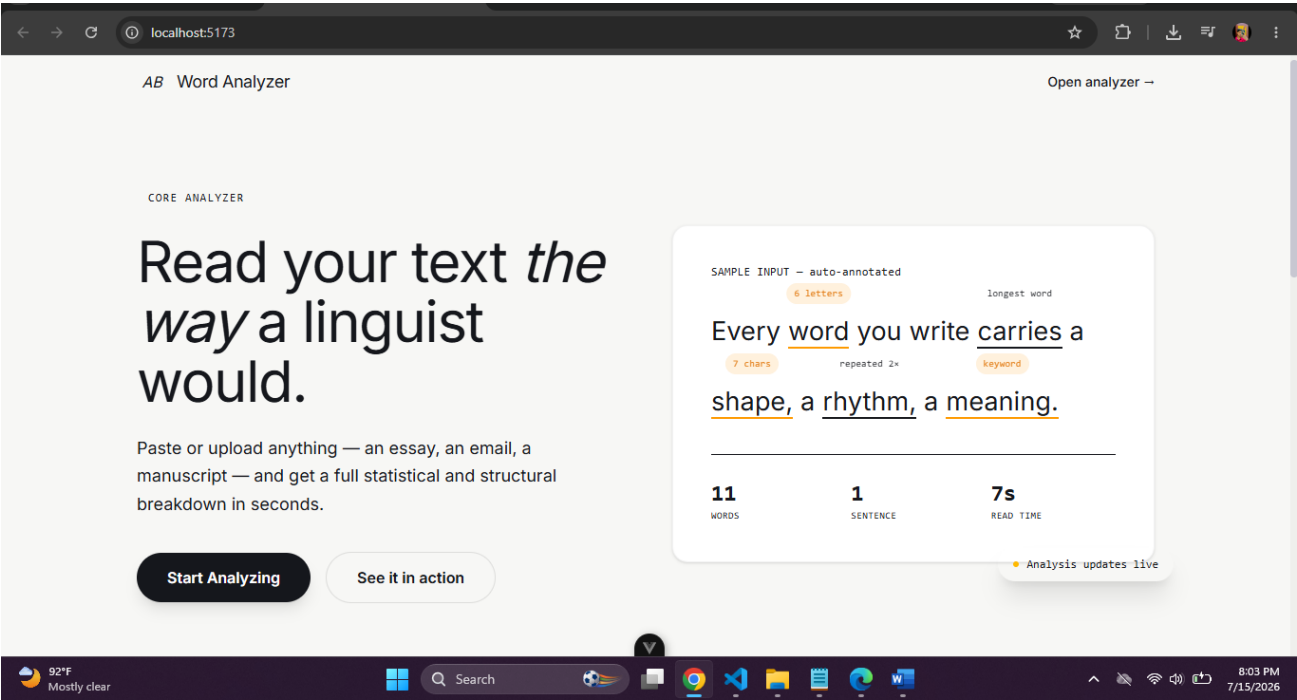
*Pinia store definition (client/stores/analyzer.ts)*

```
const props = defineProps<{ text: string }>()
```

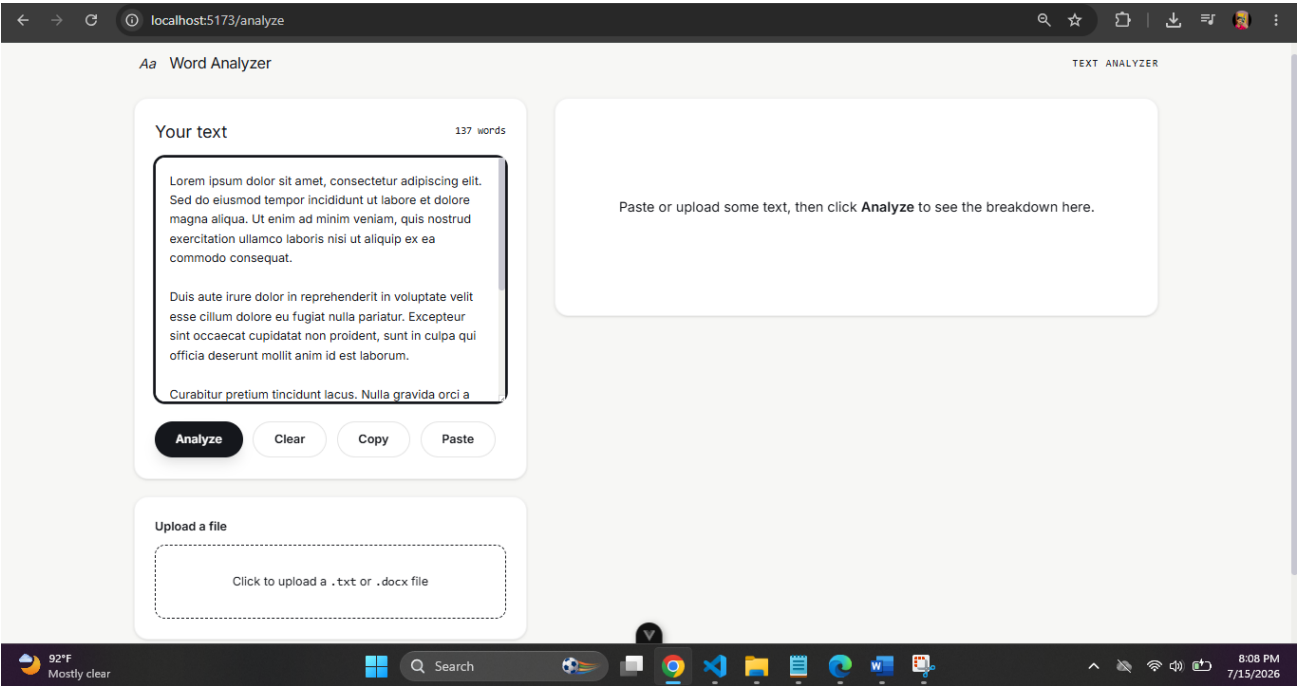
```
function run(feature: Feature) {
  store.run(feature, props.text)
}
```

*Component receiving props and dispatching a store action (client/components/AIPanel.vue)*

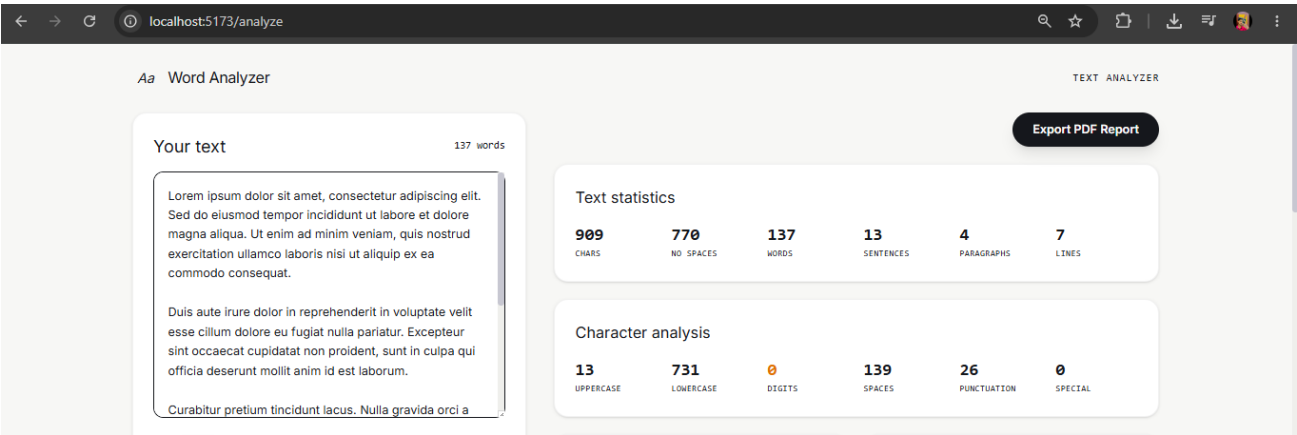
4. SCREENSHOTS



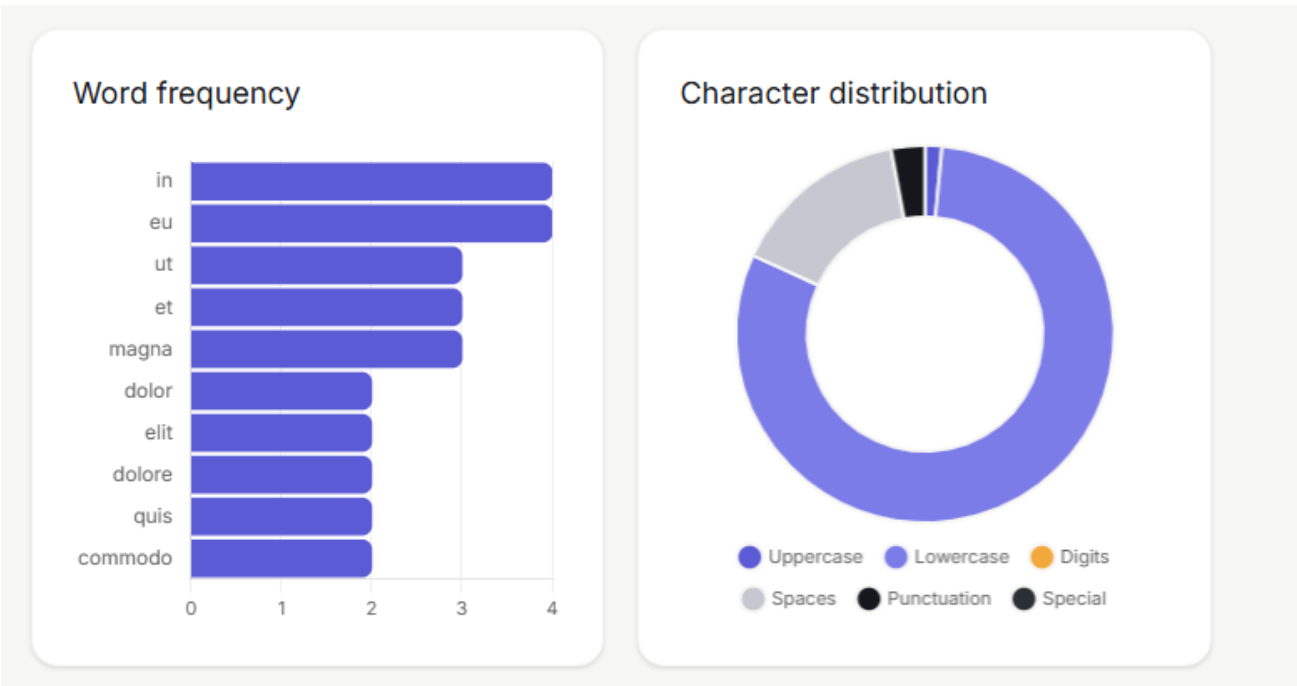
Home Page



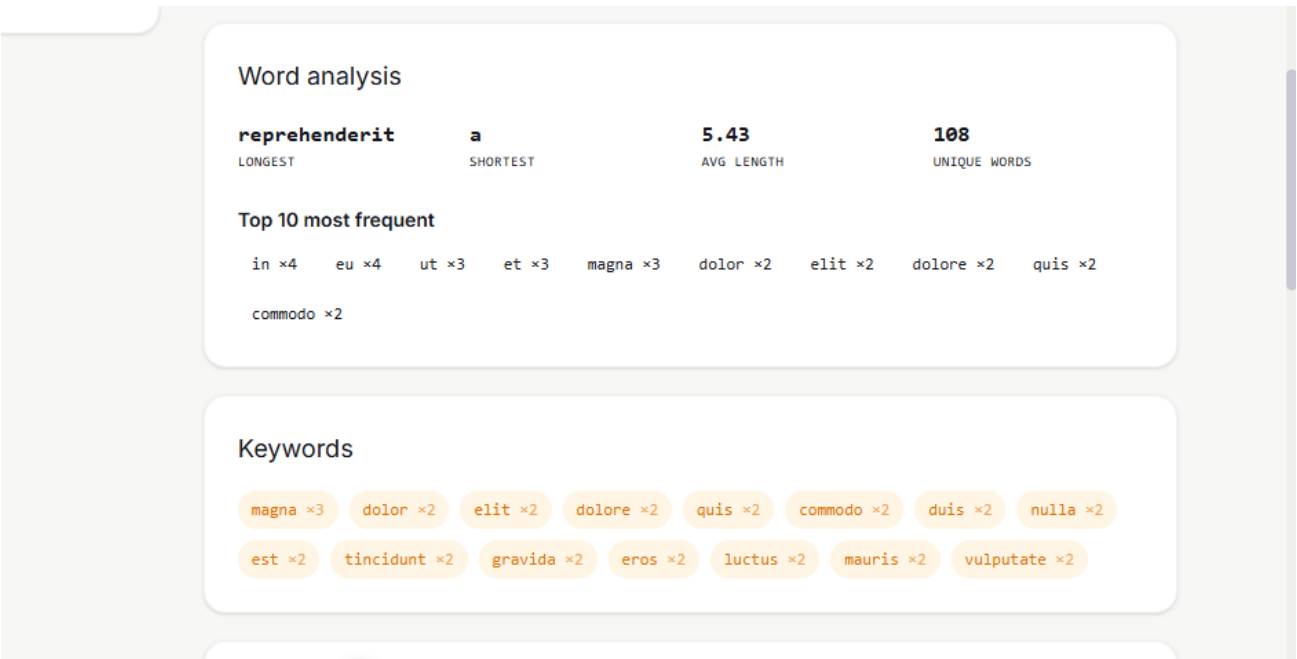
Text Input & File Upload



Text Statistics & Character Analysis



Word Frequency & Character Distribution Charts



# Reading analysis

42s

READING TIME

1m 4s

SPEAKING TIME

10.54

AVG SENTENCE LEN

5.43

AVG WORD LEN

Word Analysis, Keyword Extraction & Reading Time

AI Features

AI Summary

Concise 3-5 sentence summary

Run

Grammar Check

Issues and how to fix them

Run

Tone Detection

Formal, casual, persuasive...

Run

Improvements

Make your writing stronger

Run

Difficult Words

Hard words explained simply

Run

? Ask AI about your text

e.g. What is the main argument? Who is the audience?

Ask

AI Panel (Groq-powered features)

## Advanced Word Analyzer

Generated on 7/15/2026, 8:20:38 PM

### Text Statistics

|                        |     |
|------------------------|-----|
| Total characters       | 909 |
| Characters (no spaces) | 770 |
| Total words            | 137 |
| Total sentences        | 13  |
| Total paragraphs       | 4   |
| Total lines            | 7   |

### Character Analysis

|                    |     |
|--------------------|-----|
| Uppercase letters  | 13  |
| Lowercase letters  | 731 |
| Digits             | 0   |
| Spaces             | 139 |
| Punctuation        | 26  |
| Special characters | 0   |

### Word Analysis

|                     |               |
|---------------------|---------------|
| Longest word        | reprehenderit |
| Shortest word       | a             |
| Average word length | 5.43          |
| Total unique words  | 108           |

PDF Export

Page 7 of 13

A feedback form with a light orange background. At the top, it asks "★ How was your analysis?". Below this, there are five orange stars and the word "Excellent". A text input field contains the text "Very Fast service". At the bottom, there is a dark grey button with the text "Submit Feedback".

★ How was your analysis?

★★★★★ Excellent

Very Fast service

Submit Feedback

*Feedback Form*

## 5. ARCHITECTURE TREE

Client-side folder structure:

```
src
├── App.vue
├── assets
│   ├── base.css
│   ├── logo.svg
│   └── main.css
├── components
│   ├── AIPanel.vue
│   ├── CharacterDistributionChart.vue
│   ├── FeedbackForm.vue
│   └── WordFrequencyChart.vue
├── env.d.ts
├── main.ts
├── router
│   └── index.ts
├── services
│   ├── api.ts
│   ├── pdfExport.ts
│   └── textAnalyzer.ts
├── stores
│   ├── ai.ts
│   ├── analyzer.ts
│   └── counter.ts
└── views
    ├── AnalyzerPage.vue
    └── HomePage.vue
```

*client folder structure*



## Deliverable 2: Server-Side Architecture

This backend was developed using Express.js, MongoDB Atlas, and the Groq LLM API to support the Vue.js frontend. The backend validates incoming requests, performs all deterministic text analysis, extracts text from uploaded files, calls Groq for the AI-assisted writing features, generates PDF reports, and stores user feedback — it is the only layer of the application with credentials for the database and the AI provider.

### 1. TECHNOLOGIES USED

- **Node.js** → runtime environment for the backend application
- **Express.js** → framework used to build the REST API server and handle routing
- **MongoDB Atlas** → cloud-hosted NoSQL database used to store feedback data
- **Mongoose** → used to define schemas and interact with MongoDB
- **Multer** → handles multipart file uploads in memory before parsing
- **Mammoth** → extracts raw text from uploaded .docx files
- **PDFKit** → generates a server-side PDF export as an alternative to the client-side export
- **Groq SDK** → calls the Groq LLM API (llama-3.3-70b-versatile) for all AI-assisted writing features
- **CORS** → restricts which frontend origins may call the API
- **dotenv** → loads secrets (MONGODB\_URI, GROQ\_API\_KEY) from a local .env file

### 2. BACKEND STRUCTURE

The server contains config, controllers, models, routes, services, and utils folders alongside server.js. The config folder holds the database connection logic. Controllers receive and validate each request and call the relevant service. Services contain the actual business logic — text analysis computation and the Groq API calls. Models define MongoDB collections via Mongoose. Routes map HTTP methods and paths to controller functions. utils/stopWords.js holds a shared constant used during keyword extraction.

### 3. DATABASE CONNECTION

The application connects to MongoDB Atlas through Mongoose, with the connection string stored in an environment variable, MONGODB\_URI. The project was originally developed against a local MongoDB instance and was migrated to MongoDB Atlas specifically to support deployment, since a cloud-hosted database is reachable from Vercel while a local instance is not. A separate config/db.js file exports a connectDB() function rather than placing the connection code directly in server.js, keeping the connection logic reusable and easy to find. The function is asynchronous because mongoose.connect() returns a promise.

Because the API is deployed as a serverless function on Vercel, the connection is additionally cached on the global object across invocations, rather than being re-established on every request. Serverless functions can spin up fresh on each call, and without this caching, MongoDB Atlas's connection limit is exhausted quickly under any real traffic, causing intermittent “too many connections” failures. If the initial connection attempt fails, the promise is cleared so the next request can retry, instead of the server silently running with a broken database connection.

```

async function connectDB() {
  if (cached.conn) {
    return cached.conn
  }

  if (!cached.promise) {
    cached.promise = mongoose
      .connect(process.env.MONGODB_URI, {
        bufferCommands: false
      })
      .then((m) => {
        console.log(`MongoDB connected: ${m.connection.host}`)
        return m
      })
      .catch((err) => {
        console.error('MongoDB connection error:', err.message)
        cached.promise = null
        throw err
      })
  }
}

```

*Database connection logic (server/config/db.js)*

#### 4. DATABASE SCHEMA (MODELS)

Advanced Word Analyzer intentionally has a single, lightweight collection. The core text-analysis feature is stateless — the text a user submits is analyzed on demand and returned in the response; it is never written to the database, so no “Analysis” or “Document” model was needed.

**Feedback** → The Feedback model stores the user's rating (1–5 stars) and an optional comment after they view their analysis results, along with the word count of the text they analyzed and a createdAt timestamp. This is the only persistent data in the application, and is used to gather lightweight product feedback rather than to support any core feature.

```

const feedbackSchema = new mongoose.Schema({
  rating: {
    type: Number,
    required: true,
    min: 1,
    max: 5
  },
  comment: {
    type: String,
    trim: true,
    maxlength: 1000,
    default: ''
  },
  wordCount: {
    type: Number,
    default: 0
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
})

```

*Feedback schema (server/models/Feedback.js) / MongoDB Atlas collection view*

## 5. API ROUTES / ENDPOINTS

| Endpoint                | Method | Purpose                                                                   |
|-------------------------|--------|---------------------------------------------------------------------------|
| /api/analyze            | POST   | Run full text analysis (stats, characters, words, keywords, reading time) |
| /api/upload             | POST   | Upload a .txt or .docx file and extract its text                          |
| /api/export/pdf         | POST   | Generate a server-side PDF report (PDFKit)                                |
| /api/ai/summary         | POST   | Groq-generated 3–5 sentence summary                                       |
| /api/ai/grammar         | POST   | Groq-generated grammar issue list                                         |
| /api/ai/tone            | POST   | Groq-generated tone detection                                             |
| /api/ai/improvements    | POST   | Groq-generated writing improvement suggestions                            |
| /api/ai/difficult-words | POST   | Groq-generated plain-language explanations of hard words                  |
| /api/ai/ask             | POST   | Groq free-form Q&A scoped to the submitted text                           |
| /api/feedback           | POST   | Submit a star rating and optional comment                                 |
| /api/feedback           | GET    | Retrieve recent feedback and the average rating                           |
| /api/health             | GET    | Health check                                                              |

## 6. DEPLOYMENT ARCHITECTURE

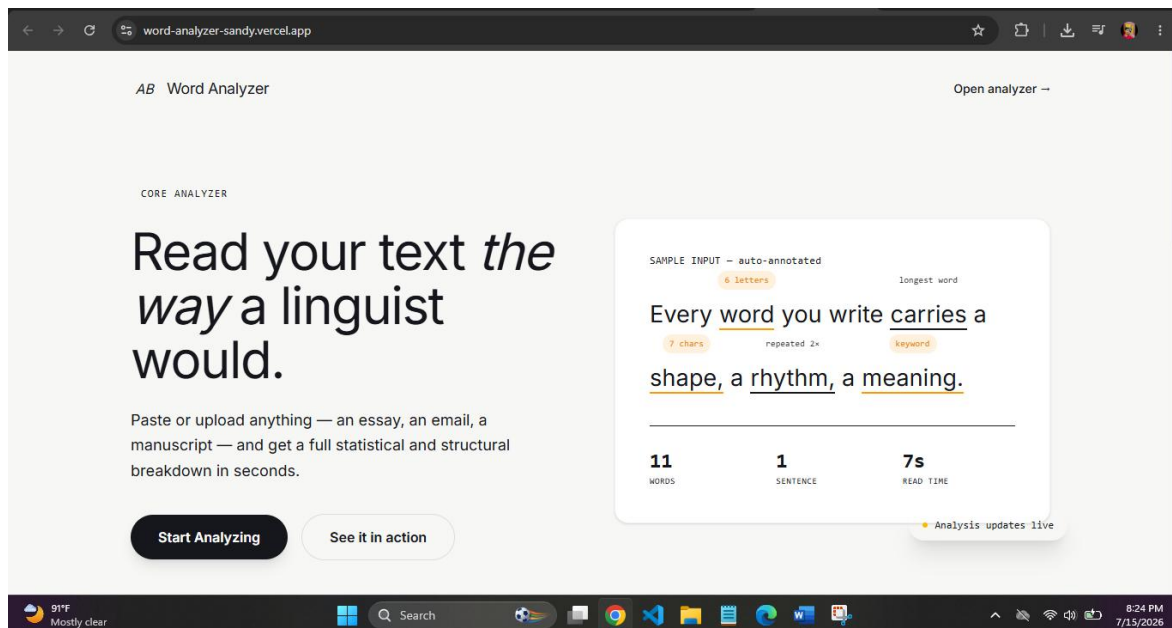
The application uses MongoDB Atlas as its database, allowing it to be reached from anywhere on the internet rather than only from localhost. The client and server are deployed as two independent services on Vercel, keeping the Express API's serverless-function configuration separate from the Vue/Vite static build.

| Concern                 | Solution                                                                                                                             |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Client (Vue/Vite build) | Deployed to Vercel; reads the API base URL from the VITE_API_URL environment variable                                                |
| Server (Express API)    | Deployed to Vercel as a Node serverless function, with the MongoDB connection cached across invocations                              |
| Database                | Migrated from a local MongoDB instance to MongoDB Atlas; network access whitelisted to 0.0.0.0/0 so Vercel's dynamic IPs can connect |
| CORS                    | Server restricts allowed origins to the deployed Vercel client domain via the CLIENT_URL environment variable                        |
| Environment variables   | MONGODB_URI, GROQ_API_KEY, and CLIENT_URL set on the server deployment; VITE_API_URL set on the client deployment                    |

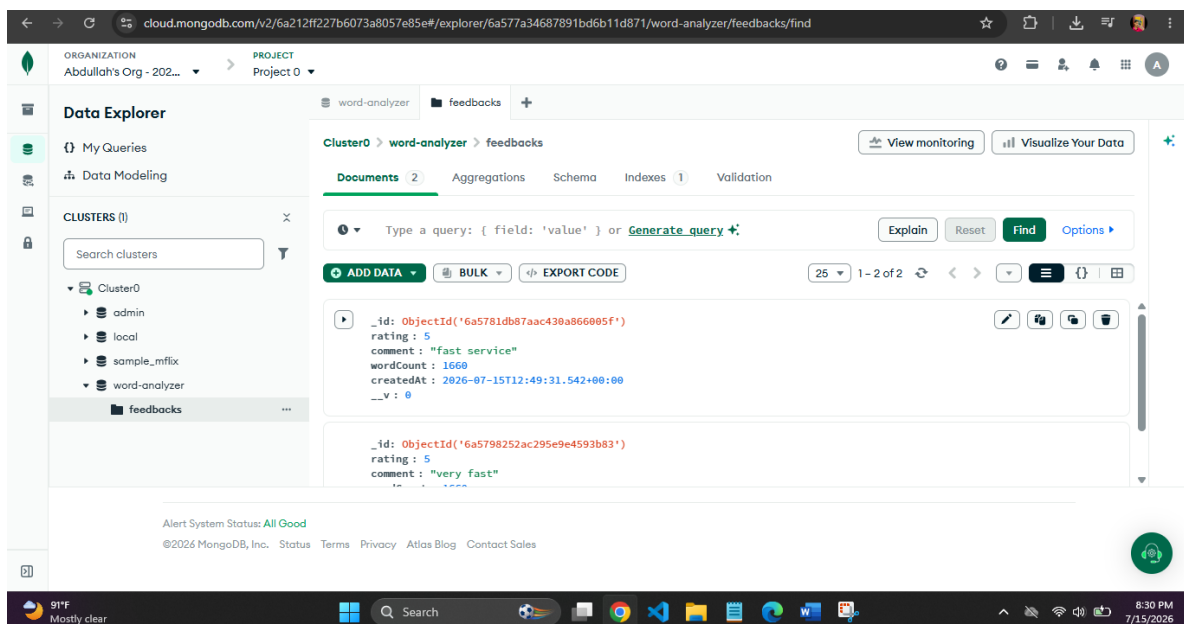
## Live Deployment

The application is live and fully functional, with both the client and server deployed on Vercel and the database hosted on MongoDB Atlas:

<https://word-analyzer-sandy.vercel.app/>



*Live deployed application (Vercel)*



*MongoDB Atlas feedback collection showing stored data*

## ARCHITECTURE TREE

Server-side folder structure:

```
server
├── config
│   └── db.js
├── controllers
│   ├── aiController.js
│   ├── analyzeController.js
│   ├── exportController.js
│   ├── feedbackController.js
│   └── uploadController.js
├── models
│   └── Feedback.js
├── package-lock.json
├── package.json
├── routes
│   ├── ai.js
│   ├── analyze.js
│   ├── export.js
│   ├── feedback.js
│   └── upload.js
├── server.js
├── services
│   ├── aiService.js
│   └── textAnalyzer.js
└── utils
    └── stopWords.js
```

*server/ folder structure*

## CONCLUSION

Advanced Word Analyzer demonstrates a complete full-stack workflow: a reactive Vue 3 frontend with centralized Pinia state management, a modular Express REST API, cloud-hosted MongoDB Atlas storage, and integration with a third-party LLM provider (Groq) for AI-assisted writing feedback. The client and server were deployed as independent services to reflect a production-realistic split architecture and to avoid the pitfalls of running a persistent, stateful database connection inside a serverless function without proper connection caching. Overall, the project meets its objectives of providing instant statistical text analysis alongside generative AI features in a single, clean interface. Future work could include user accounts with saved analysis history, multi-language support, and wiring up the existing server-side PDF export as a fallback option.